

Python

Syntaxe · types · collections · contrôle · fonctions · objets · exceptions · fichiers · bibliothèque standard

print(x) chasse fixe, gras : code à taper tel quel
nom italique : valeur à remplacer
→ **42** résultat obtenu, vérifié en exécutant l'exemple
↩ retour à la ligne dans ce qui est affiché
3.13+ version minimale de Python requise
❗ piège fréquent

LANCER PYTHON

python	ouvrir l'interpréteur interactif ; Windows : aussi py ; Linux, macOS : souvent python3
python prog.py	exécuter un programme
python -i prog.py	exécuter, puis rester dans l'interpréteur pour inspecter les variables
python -m module	lancer un module : pip , venv , http.server ...
python -c "code"	exécuter une seule ligne
python -V exit()	afficher la version installée quitter l'interpréteur ; 3.13+ exit suffit ; ou Ctrl+D (Windows : Ctrl+Z) puis Entrée)

WINDOWS : GESTIONNAIRE D'INSTALLATION **3.13+**

py install 3.14	installer une version
py list	lister les versions installées
py -V:3.14 prog.py	lancer avec une version précise
py help	toutes les commandes

L'installateur Windows classique est déprécié depuis 3.14 ; il ne sera plus produit à partir de 3.16.

L'INTERPRÉTEUR INTERACTIF

- **>>>** invite ; ... suite d'un bloc — une ligne vide le termine.
- **_** rappelle le dernier résultat affiché.
- **↑ ↓** historique ; **Tab** complète les noms.
- **help(len)** · **help("for")** : aide ; **dir(x)** : attributs de **x** ; **type(x)** : son type.
- **3.13+** couleurs, **F1** aide, **F2** historique, **F3** mode collage.

ANATOMIE D'UN PROGRAMME

```
"""Aire d'un disque.""" ❶
import math              ❷

RAYON = 2.5              ❸

def aire(r):              ❹
    """Renvoie l'aire du disque."""
    return math.pi * r ** 2 ❺

if __name__ == "__main__": ❻
    print(f"Aire(RAYON): {aire(RAYON):.2f}") → 19.63
```

- ❶ **docstring** : documentation du module, lue par **help()** ;
- ❷ **import** d'un module de la bibliothèque standard ;
- ❸ **constante** : nom en majuscules (simple convention) ;
- ❹ **définition de fonction** : **def**, nom, paramètres, deux-points ;
- ❺ le **bloc** est délimité par l'indentation : 4 espaces ;
- ❻ exécuté seulement si le fichier est **lancé**, pas s'il est **importé**.

SYNTAXE DE BASE

- Un bloc s'ouvre par **:** et se délimite par l'**indentation** : 4 espaces, sans mélange avec des tabulations.
- Une instruction par ligne, sans **;** final.
- **#** commente jusqu'à la fin de la ligne.
- Ligne trop longue : la couper à l'intérieur de **()**, **[]** ou **{ }** plutôt qu'avec ****.
- La casse compte : **nom ≠ Nom**.
- **pass** : bloc vide, à compléter plus tard.

VARIABLES ET AFFECTATION

Une variable est un **nom** attaché à un objet. Le type appartient à l'objet, pas au nom : c'est le **typage dynamique**.

age = 36	créer, ou réaffecter
a, b = 1, 2	(1, 2) · affectation multiple
a, b = b, a	(2, 1) · échange, sans variable temporaire
x = y = 0	(0, 0) · même valeur
x += 5	5 · équivaut à x = x + 5 ; de même -= *= /= ...
type(3.5)	<code><class 'float'></code> · type de l'objet
isinstance(5, int) → True	· test de type
del x	supprimer le nom

NOMMAGE (PEP 8)

nombre_total	variable, fonction : <i>snake_case</i>
CompteBancaire	classe : <i>CapWords</i>
TAUX_MAX	constante
_interne	usage interne (convention)
__init__	méthode spéciale, dite « dunder »
_	valeur ignorée : for _ in range(3):

Un nom : lettres, chiffres et **_**, sans chiffre en tête ni mot réservé.

LES 35 MOTS RÉSERVÉS

Valeurs	False None True
Logique	and or not is in
Conditions	if elif else
Boucles	for while break continue
Fonctions	def return lambda yield global nonlocal
Classes	class
Exceptions	try except finally raise assert
Modules	import from as
Divers	with pass del
Asynchrone	async await
Contextuels	match case _ type · réservés selon le contexte

OPÉRATEURS

ARITHMÉTIQUE

7 / 2	3.5 · division : toujours un float
7 // 2	3 · division entière (quotient)
-7 // 2	-4 ❗ · arrondi vers le bas
7 % 3	1 · reste (modulo)
2 ** 10	1024 · puissance
-2 ** 2	-4 ❗ · ** passe avant le signe

COMPARAISON ET LOGIQUE

3 == 3.0	True · égalité de valeur
0 < 5 <= 10	True · comparaisons enchaînées
x is None	True · identité : pour None
"ch" in "chat"	True · appartenance
not 0	True · négation
0 or "défaut"	'défaut' · 1 ^{er} opérande vrai
"" and 1 / 0	' ' · court-circuit : 1 / 0 jamais évalué

BIT À BIT

6 & 3, 6 3, 6 ^ 3 → (2, 7, 5)	· et, ou, ou exclusif
1 << 4, ~5	(16, -6) · décalage, inversion

PRIORITÉ, DE LA PLUS FORTE À LA PLUS FAIBLE

(), ******, **+x -x ~x**, *** / // %**, **+ -**,
<< >>, **&**, **^**, **|**, **== < in is...**, **not**,
and, **or**, **a if c else b**, **lambda**, **:=**

Dans le doute : des parenthèses. **2 + 3 * 4 → 14** ;
(2 + 3) * 4 → 20.

AFFICHER ET SAISIR

```
nom, age = "Alice", 36
print("Bonjour", nom)
→ Bonjour Alice
print(nom, age, sep=" | ")
→ Alice | 36
print(f"{nom} a {age} ans")
→ Alice a 36 ans
print(f"age={age}")          → age=36
print("fin", end="!\n")     → fin!

texte = input("Âge ? ") # une chaîne
age = int(texte)         # conversion

input() renvoie du texte : convertir avec int() ou float() avant de calculer.
```

VRAI OU FAUX ?

Valent **faux** : **False, None, 0, 0.0, "", [], (), {}, set(), range(0)**. Tout le reste vaut **vrai**.

bool([]) **False** · liste vide

bool("0") **True** ❗ · chaîne non vide

bool(" ") **True** ❗ · une espace n'est pas rien

D'où l'usage : **if liste** : plutôt que **if len(liste) > 0** :

2 · Nombres et chaînes de caractères

MÉMENTO — PYTHON

TYPES DE BASE	
<code>int</code>	<code>42 - 7 1_000_000 0xFF 0b1010</code> — entiers, sans limite de taille
<code>float</code>	<code>3.14 1e-3 float("inf")</code> — à virgule flottante, ≈ 15 chiffres significatifs
<code>complex</code>	<code>2+3j — z.real, z.imag</code>
<code>bool</code>	<code>True False</code> — sous-type de <code>int</code>
<code>str</code>	<code>'a' "a" "" "a" ""</code> — texte Unicode, immuable
<code>bytes</code>	<code>b"abc"</code> — octets bruts (fichiers binaires, réseau)
<code>NoneType</code>	<code>None</code> — absence de valeur
<code>2 ** 100</code>	→ 1267650600228229401496703205376
<code>0xFF, 0b1010, 1_000</code>	→ (255, 10, 1000) · hexadécimal, binaire, séparateur
<code>True + True</code>	<code>2 · True</code> vaut 1

CALCULER	
<code>abs(-3.5)</code>	3.5 · valeur absolue
<code>divmod(17, 5)</code>	(3, 2) · quotient, reste
<code>round(3.14159, 2)</code>	3.14
<code>round(2.5), round(3.5)</code>	→ (2, 4) ❶ · arrondi au pair le plus proche
<code>max(3, 7, 1)</code>	7 · et min
<code>sum([1, 2, 3])</code>	6
<code>pow(2, 10, 1000)</code>	24 · 2 ¹⁰ modulo 1000
MODULE MATH (IMPORT MATH)	
<code>math.sqrt(2)</code>	→ 1.4142135623730951
<code>math.floor(-2.5), math.ceil(2.1)</code>	→ (-3, 3) · vers le bas, vers le haut
<code>math.gcd(12, 18), math.lcm(4, 6)</code>	→ (6, 12) · PGCD, PPCM
<code>math.factorial(5), math.comb(5, 2)</code>	→ (120, 10) · 5 !, combinaisons
<code>math.log10(1000), math.log2(8)</code>	→ (3.0, 3.0) · logarithmes ; <code>math.log(x)</code> : népérien
<code>math.dist((0, 0), (3, 4))</code>	→ 5.0 · distance
<code>math.pi · math.e</code>	· constantes
<code>math.inf</code>	

LE PIÈGE DES FLOTTANTS	
<code>0.1 + 0.2</code>	→ <code>0.30000000000000004</code>
<code>0.1 + 0.2 == 0.3</code>	→ <code>False</code> ❶
<code>import math</code>	
<code>math.isclose(0.1 + 0.2, 0.3)</code>	→ <code>True</code>
<code>from decimal import Decimal</code>	
<code>Decimal("0.1") + Decimal("0.2")</code>	→ <code>Decimal('0.3')</code>
<code>from fractions import Fraction</code>	
<code>Fraction(1, 3) + Fraction(1, 6)</code>	→ <code>Fraction(1, 2)</code>

Les `float` sont stockés en binaire : 0,1 n'y a pas de valeur exacte. Comparer avec `math.isclose` ; pour de l'argent, `Decimal` construit à partir d'une chaîne (`Decimal(0.1)` hérite de l'erreur).

CHAÎNES DE CARACTÈRES	
<code>"l'été"</code> · <code>'dit "oui"'</code>	guillemets simples ou doubles, au choix
<code>""" ... """</code>	chaîne sur plusieurs lignes
<code>\n \t \\ \\"</code> <code>\u00e9</code>	saut de ligne, tabulation, \, ", caractère par son code (é)
<code>r"C:\temp\new"</code>	chaîne brute : \ non interprété (chemins Windows, expressions régulières)
<code>"ab" + "cd"</code>	<code>'abcd'</code> · concaténation
<code>"ab" * 3</code>	<code>'ababab'</code> · répétition
<code>len("été")</code>	3 · longueur en caractères
<code>"Z" < "a"</code>	<code>True</code> ❶ · ordre des codes : majuscules d'abord
<code>s[0] = "j"</code>	<code>TypeError</code> · immuable : on crée une nouvelle chaîne

MÉTHODES DE CHAÎNES	
Elles renvoient une nouvelle chaîne : <code>s.upper()</code> ne modifie pas s.	
<code>"été".upper()</code>	<code>'ÉTÉ'</code> · aussi <code>lower()</code>
<code>"le petit prince".title()</code>	→ <code>'Le Petit Prince'</code>
<code>"bonjour".capitalize()</code>	→ <code>'Bonjour'</code>
<code>" a b ".strip()</code>	<code>'a b '</code> · aussi <code>lstrip, rstrip</code>
<code>"a,b,,c".split(",")</code>	→ <code>['a', 'b', '', 'c']</code>
<code>"un deux\ttrois".split()</code>	→ <code>['un', 'deux', 'trois']</code> · sans argument : tout blanc
<code>" - ".join(["a", "b", "c"])</code>	→ <code>'a - b - c'</code> · inverse de <code>split</code>
<code>"chat".replace("a", "o")</code>	→ <code>'chot'</code>
<code>"banane".count("an")</code>	2
<code>"banane".find("n")</code>	2 · -1 si absent
<code>"banane".index("z")</code>	<code>ValueError</code> · si absent
<code>"img.png".endswith(".png")</code>	→ <code>True</code> · et <code>startswith</code>
<code>"img.png".removesuffix(".png")</code>	→ <code>'img'</code> · ❸ et <code>removeprefix</code>
<code>"clé=val".partition("=")</code>	→ <code>('clé', '=', 'val')</code>
<code>"7".zfill(3)</code>	<code>'007'</code>
<code>"titre".center(9, "*")</code>	→ <code>***titre**</code> · et <code>ljust, rjust</code>
<code>"123".isdigit()</code>	<code>True</code> · et <code>isalpha, isspace...</code>
<code>"Straße".casefold()</code>	<code>'strasse'</code> · comparer sans la casse

CONVERSIONS	
<code>int("42")</code>	42
<code>int(3.99)</code>	3 · troncature vers zéro
<code>int("ff", 16)</code>	255 · lecture en base 16
<code>float("2.5")</code>	2.5
<code>float("2,5")</code>	<code>ValueError</code> ❶ · virgule décimale refusée
<code>int("abc")</code>	<code>ValueError</code> · texte non numérique
<code>str(42)</code>	<code>'42'</code>
<code>ord("é"), chr(65)</code>	→ (233, 'A') · code Unicode ↔ caractère
<code>hex(255), bin(5)</code>	(<code>'0xff'</code> , <code>'0b101'</code>)
<code>list("abc")</code>	[<code>'a'</code> , <code>'b'</code> , <code>'c'</code>]

METTRE EN FORME : F-STRINGS		
<code>f"{ valeur !r : >10.2f }"</code>	EXPRESSION	CONVERSION FORMAT
Après :, dans l'ordre : remplissage, alignement (< ^ >), signe, largeur, séparateur (, ou _), précision (.2), type (f d % e x b).		
<code>f"{x:.2f}"</code>		<code>'3.14'</code> · 2 décimales
<code>f"{x:8.3f}"</code>		<code>' 3.142'</code> · largeur 8
<code>f"{n:,}"</code>		<code>'1,234,567'</code> · milliers
<code>f"{n:,}".replace(",","")</code>		→ <code>'1 234 567'</code> · à la française
<code>f"{0.256:.1%}"</code>		<code>'25.6%'</code> · pourcentage
<code>f"{42:05d}"</code>		<code>'00042'</code> · zéros en tête
<code>f"{nom:<6} "</code>		<code>'Ada '</code> · à gauche
<code>f"{nom:>6} "</code>		<code>' Ada'</code> · à droite
<code>f"{nom:^7}"</code>		<code>***Ada**</code> · centré, remplissage
<code>f"{255:x}", f"{5:08b}"</code>		→ (<code>'ff'</code> , <code>'00000101'</code>) · hexadécimal, binaire
<code>f"{nom!r}"</code>		<code>''Ada''</code> · avec <code>repr()</code>
<code>f"{x:=.1f}"</code>		<code>'x=3.1'</code> · débogage ❸
<code>f"{{x}}"</code>		<code>'{x}'</code> · accolades littérales

Anciennes syntaxes, encore rencontrées : `"{:2f}".format(x) et "%.2f" % x. ❸ t"`... crée un gabarit (**Template**), pas une chaîne : il sert aux bibliothèques qui doivent traiter les valeurs insérées.

TEXTE, OCTETS, ENCODAGES	
<code>str</code> : des caractères ; fichiers et réseau : des octets (<code>bytes</code>).	
<code>"é".encode("utf-8")</code>	<code>b'\xc3\xa9'</code> · texte → octets
<code>b"\xc3\xa9".decode("utf-8")</code>	→ <code>'é'</code> · octets → texte
<code>len("é"), len("é".encode())</code>	→ (1, 2) · 1 caractère, 2 octets
<code>"\u00e9" == "e\u0301"</code>	<code>False</code> ❶ · é précomposé ≠ e + accent
<code>unicodedata.normalize("NFC", "e\u0301") == "é"</code>	→ <code>True</code> · normaliser avant de comparer

3 · Collections : listes, dictionnaires, ensembles, tuples

MÉMENTO — PYTHON

CHOISIR SA COLLECTION

Type	Écriture	Ordre	Modif.	Doubl.
list	[1, 2, 2]	oui	oui	oui
tuple	(1, 2, 2)	oui	non	oui
dict	{"a": 1}	oui*	oui	clés : non
set	{1, 2}	non	oui	non
str	"abc"	oui	non	oui

* ordre d'insertion, garanti depuis 3.7. Modif. : modifiable ; Doubl. : doublons admis. **frozenset** : ensemble non modifiable, donc hachable.

- **list** : suite d'éléments qu'on ajoute, retire, trie.
- **tuple** : enregistrement fixe (coordonnées, retour multiple) ; peut servir de clé.
- **dict** : retrouver une valeur par sa clé, en temps constant.
- **set** : tester l'appartenance, dédoubler, comparer des groupes.

❶ {} est un dict vide ; ensemble vide : **set()**. Tuple d'un seul élément : (1,) — c'est la virgule qui fait le tuple.

LISTES

Chaque ligne part de **L** = [3, 1, 4].

L.append(1)	[3, 1, 4, 1] · ajouter à la fin
L.extend([5, 9])	→ [3, 1, 4, 5, 9] · ajouter plusieurs
L.insert(0, 7)	[7, 3, 1, 4] · insérer à l'indice 0
L.pop()	4 · retire et renvoie le dernier
L.pop(0)	3 · ... ou celui de l'indice 0
L.remove(1)	[3, 4] · 1 ^{re} occurrence ; ValueError si absente
L[1] = 9	[3, 9, 4] · remplacer
del L[0]	[1, 4] · supprimer par indice
L[1:] = []	[3] · remplacer une tranche
L.index(4)	2 · position
L.sort()	[1, 3, 4] · tri sur place , renvoie None
sorted(L)	[1, 3, 4] · nouvelle liste, L intacte
L.sort(reverse=True)	→ [4, 3, 1] · ordre décroissant
L.reverse()	[4, 1, 3] · inverser
L = L.sort()	None ❶ · piège classique
L + [1, 5]	[3, 1, 4, 1, 5] · concaténer
[0] * 3	[0, 0, 0]
4 in L, len(L), sum(L)	→ (True, 3, 8) · et min, max
L.copy()	[3, 1, 4] · copie (ou L[:])

TRIÉ AVEC UNE CLÉ

```
mots = ["Kiwi", "ail", "pomme", "figue"]
sorted(mots) # A-Z avant a-z
→ ['Kiwi', 'ail', 'figue', 'pomme'] ❶
sorted(mots, key=str.lower)
→ ['ail', 'figue', 'Kiwi', 'pomme']
sorted(mots, key=len)
→ ['ail', 'Kiwi', 'pomme', 'figue']
```

Le tri est **stable** : à clé égale, l'ordre d'origine est conservé (pomme avant figue).

DICTIONNAIRES

Chaque ligne part de **d** = {"a": 1, "b": 2}.

d["a"]	1 · lire
d["z"]	KeyError ❶ · clé absente
d.get("z")	None · sans erreur
d.get("z", 0)	0 · valeur par défaut
d["c"] = 3	→ {'a': 1, 'b': 2, 'c': 3} · ajoute ou remplace
del d["a"]	{'b': 2}
d.pop("a")	1 · retire et renvoie
"a" in d	True · teste les clés
list(d)	['a', 'b'] · les clés
list(d.values())	[1, 2]
list(d.items())	→ [('a', 1), ('b', 2)]
d {"b": 9}	{'a': 1, 'b': 9} · fusion [3.9+]
d.update(c=3)	→ {'a': 1, 'b': 2, 'c': 3} · fusion sur place
d.setdefault("l", []).append(5)	→ {'a': 1, 'b': 2, 'l': [5]} · crée si absent
dict(zip("xy", [1, 2]))	→ {'x': 1, 'y': 2} · à partir de deux listes
dict.fromkeys("xy", 0)	→ {'x': 0, 'y': 0}
{1: "x"}	TypeError ❶ · une liste ne peut être clé
for cle, valeur in d.items(): print(cle, valeur)	→ a 1 b 2

Clés : objets **hachables**, donc immuables (**str**, **int**, **tuple**...).

ENSEMBLES

Avec **a** = {1, 2, 3} et **b** = {3, 4} :

a b	{1, 2, 3, 4} · union
a & b	{3} · intersection
a - b	{1, 2} · différence
a ^ b	{1, 2, 4} · différence symétrique
{1, 2} <= a	True · inclusion
a.add(9)	{1, 2, 3, 9} · ajouter
a.discard(7)	{1, 2, 3} · retirer sans erreur
a.remove(7)	KeyError · si absent
set([3, 1, 3, 2])	{1, 2, 3} · dédoubler
list(dict.fromkeys([3, 1, 3]))	→ [3, 1] · ... en gardant l'ordre

Pas d'indice, ordre non garanti ; éléments hachables seulement.

INDICES ET TRANCHES

	0	1	2	3	4	5	6
s =	P	Y	T	H	O	N	
	-6	-5	-4	-3	-2	-1	

Les indices désignent les **bords** des cases ; comptés depuis la fin, ils sont négatifs. Syntaxe : **s[début:fin:pas]**, fin **exclue**.

s[0]	'P' · premier
s[-1]	'N' · dernier
s[1:4]	'YTH' · de 1 inclus à 4 exclu
s[:2]	'PY' · les 2 premiers
s[2:]	'THON' · à partir de l'indice 2
s[-3:]	'HON' · les 3 derniers
s[::2]	'PTO' · un sur deux
s[::-1]	'NOHTYP' · à l'envers
s[10]	IndexError ❶ · hors limites...
s[2:10]	'THON' · ... mais une tranche tolère

Valable pour toute séquence : chaînes, listes, tuples.

TUPLES ET DÉBALLAGE

```
point = (3, 4)
x, y = point # déballage
→ 3
premier, *reste = [1, 2, 3, 4]
reste → [2, 3, 4]
*debut, dernier = "abc"
debut → ['a', 'b']
a, _ = (10, "x") # _ : ignoré
def bornes(L):
    return min(L), max(L) # un tuple
bas, haut = bornes([4, 1, 7])
→ (1, 7)
```

COMPRÉHENSIONS

[x * 2 for x in L if x > 0]

EXPRESSION BOUCLE FILTRE (FACULTATIF)

```
[x ** 2 for x in range(5)]
→ [0, 1, 4, 9, 16]
[x for x in range(10) if x % 3 == 0]
→ [0, 3, 6, 9]
[x if x > 0 else 0 for x in [-2, 5, -1]]
→ [0, 5, 0]
{m: len(m) for m in ["un", "deux"]}
→ {'un': 2, 'deux': 4}
{x % 3 for x in range(10)}
→ {0, 1, 2}
sum(x ** 2 for x in range(4)) → 14
["{i}{j}" for i in "ab" for j in "12"]
→ ['a1', 'a2', 'b1', 'b2']
```

if après for : filtre. ... if ... else ... avant for : transforme chaque élément. Sans crochets, entre parenthèses : une **expression génératrice**, calculée à la demande, sans liste en mémoire.

RÉFÉRENCES ET COPIES

```
a = [1, 2]
b = a # deux noms, UN objet
b.append(3)
a → [1, 2, 3] ❶
c = a.copy() # ou list(a), a[:]:
c == a, c is a → (True, False)
m = [[0] * 2] * 2 # 2 fois la même ligne
m[0][0] = 1
m → [[1, 0], [1, 0]] ❶
# correct : une ligne par tour
m = [[0] * 2 for _ in range(2)]
import copy
m2 = copy.deepcopy(m) # copie profonde
```

== compare les valeurs, **is** l'identité. **Immuables** : **int**, **float**, **str**, **tuple**, **frozenset**, **bytes**. **Modifiables** : **list**, **dict**, **set**, la plupart des objets. **copy()** ne copie qu'un niveau.

4 · Conditions, boucles, itération

MÉMENTO — PYTHON

CONDITIONS

```
note = 13
if note >= 16:
    mention = "très bien"
elif note >= 14:
    mention = "bien"
elif note >= 12:
    mention = "assez bien"
else:
    mention = "aucune"
mention → 'assez bien'
age = 20
s = "majeur" if age >= 18 else "mineur"
s → 'majeur'
```

TESTS IDIOMATIQUES

if x:	x non vide, non nul
if not liste:	liste vide
if x is None:	jamais == None
if 0 <= n < 10:	encadrement
if rep in ("o", "oui"):	plusieurs valeurs
if isinstance(x, (int, float)):	type

MATCH : FILTRAGE PAR MOTIF 3.10+

```
def analyser(cmd):
    match cmd.split():
        case ["aller", direction]:
            return f"vers {direction}"
        case ["prendre", *obj]:
            return f"{len(obj)} objets"
        case ["quitter" | "q"]:
            return "fin"
        case [n] if n.isdigit():
            return f"code {n}"
        case _:
            return "inconnu"

analyser("aller nord") → 'vers nord'
analyser("prendre a b") → '2 objets'
analyser("q") → 'fin'
analyser("42") → 'code 42'
analyser("") → 'inconnu'
```

- Motifs : littéral **case 0:**, alternative **|**, capture **x**, séquence **[a, *reste]**, dictionnaire **{"type": t}**, classe **Point(x=0)**, garde **if**.
- **case _:** attrape tout le reste.
- **!** Un nom seul **capture**, il ne compare pas : **case ROUGE:** attrape tout. Comparer à une constante : **nom pointé**, **case Couleur.ROUGE:**

OPÉRATEUR MORSE := 3.8+

```
mots = "un deux trois quatre".split()
# affecte n, puis le compare
if (n := len(mots)) > 3:
    print(n, "mots") → 4 mots
```

Utile dans **while (ligne := f.readline()):**; à réserver aux cas où il évite une répétition.

REDEMANDER JUSQU'À UNE SAISIE VALIDE

```
while True:
    texte = input("Note (0-20) ? ")
    try:
        note = int(texte)
    except ValueError:
        print("Un entier, svp.")
        continue
    if 0 <= note <= 20:
        break
    print("Hors limites.")
```

BOUCLE FOR

```
for lettre in "abc":
    print(lettre, end=" ") → a b c

list(range(5)) → [0, 1, 2, 3, 4] · 5 exclu
list(range(2, 6)) → [2, 3, 4, 5]
list(range(0, 10, 3)) → [0, 3, 6, 9] · pas de 3
list(range(3, 0, -1)) → [3, 2, 1] · à rebours
```

OUTILS DE PARCOURS

```
list(enumerate("ab", 1)) → [(1, 'a'), (2, 'b')] · numéro + élément
list(zip("ab", [1, 2, 3])) → [('a', 1), ('b', 2)] · s'arrête au plus court; strict=True 3.10+ : erreur si longueurs différentes
list(reversed([1, 2, 3])) → [3, 2, 1]
any(x > 2 for x in [1, 3]) → True · au moins un
all(x > 2 for x in [1, 3]) → False · tous
list(map(str.upper, "ab")) → ['A', 'B'] · appliquer
list(filter(str.isdigit, "a1b2")) → ['1', '2'] · filtrer
max(["kiwi", "ananas"], key=len) → 'ananas' · selon un critère
```

```
noms = ["Ana", "Léo"]
notes = [15, 12]
for nom, note in zip(noms, notes):
    print(nom, note) → Ana 15 → Léo 12
```

WHILE, BREAK, CONTINUE, ELSE

```
n = 0
while n < 3:
    n += 1
    # tant que c'est vrai
    # sinon : sans fin
    n → 3
for x in [4, 9, 15, 7]:
    if x % 2 == 0:
        continue # tour suivant
    if x > 10:
        break
    print(x) → 9
else:
    print("fini") # si aucun break
```

- Boucle sans fin volontaire : **while True:** ...
- **Ctrl+C** interrompt un programme bloqué (**KeyboardInterrupt**).
- **!** Ne pas modifier une liste pendant qu'on la parcourt : parcourir une copie (**for x in L[:]**) ou construire une nouvelle liste.

PIÈGES COURANTS

```
x == 1 or 2 → 2 ! · toujours vrai ! → x in (1, 2)

list(range(1, 4)) → [1, 2, 3] · fin exclue : 1 à 10 → range(1, 11)
0.1 + 0.2 == 0.3 → False ! · flottants : math.isclose
for i in range(len(L)): fonctionne, mais
range(len(L)): for x in L: ou enumerate(L) est plus clair

if x = 5: → SyntaxError := affecte, == compare.
```

RECETTES COURANTES

```
L = [3, 1, 3, 2]
d = {"a": 3, "b": 7, "c": 5}
M = [[1, 2], [3, 4]]
# compter les occurrences
from collections import Counter
Counter("abracadabra").most_common(2) → [('a', 5), ('b', 2)]
# clé de la plus grande valeur
max(d, key=d.get) → 'b'
# trier un dict par valeur
sorted(d.items(), key=lambda kv: kv[1]) → [('a', 3), ('c', 5), ('b', 7)]
# inverser un dict
{v: k for k, v in d.items()} → {3: 'a', 7: 'b', 5: 'c'}
# aplatir une liste de listes
[x for ligne in M for x in ligne] → [1, 2, 3, 4]
# transposer une matrice
list(zip(*M)) → [(1, 3), (2, 4)]
# dédoubler en gardant l'ordre
list(dict.fromkeys(L)) → [3, 1, 2]
# compter à la main
cpt = {}
for c in "abca":
    cpt[c] = cpt.get(c, 0) + 1
cpt → {'a': 2, 'b': 1, 'c': 1}
# indice du maximum
L.index(max(L)) → 0
# découper en paquets (3.12+)
from itertools import batched
list(batched("abcde", 2)) → [('a', 'b'), ('c', 'd'), ('e',)]
```

MOTIFS DE BOUCLE

```
notes = [12, 8, 15, 9]
total = 0 # accumulateur
for n in notes:
    total += n
total → 44
nb = 0 # compteur
for n in notes:
    if n >= 10:
        nb += 1
nb → 2
m = None # meilleur vu
for n in notes:
    if m is None or n > m:
        m = n
m → 15
```

Formes courtes : **sum(notes)**, **sum(n >= 10 for n in notes)**, **max(notes)**.

ITÉRATEURS ET GÉNÉRATEURS

```
it = iter([10, 20])
next(it), next(it) → (10, 20)
next(it) → StopIteration

def rebours(n):
    while n > 0:
        yield n
        n -= 1
list(rebours(3)) → [3, 2, 1]
# expression génératrice :
carres = (x * x for x in range(10**9))
next(carres), next(carres) → (0, 1)
```

for appelle **iter()**, puis **next()** jusqu'à **StopIteration**. Un générateur fournit ses valeurs à la demande : mémoire constante, mais un seul parcours. **yield from** délègue à un autre itérable.

5 · Fonctions et modules

MÉMENTO — PYTHON

DÉFINIR ET APPELER UNE FONCTION

```
def prix_ttc(ht, taux=0.20):  
    """Renvoie le prix TTC arrondi au centime."""  
    return round(ht * (1 + taux), 2)  
  
prix_ttc(100)           → 120.0  
prix_ttc(100, 0.055)   → 105.5  
prix_ttc(taux=0.1, ht=50) → 55.0
```

- 1 **def**, nom, paramètres ; **taux** a une **valeur par défaut** ;
- 2 **docstring** : ce que fait la fonction, affichée par **help(prix_ttc)** ;
- 3 **return** renvoie la valeur et **termine** la fonction ; sans **return**, elle renvoie **None**.

À l'appel : arguments **positionnels** (par rang) d'abord, puis **nommés** (**taux=0.1**), dans n'importe quel ordre.

PARAMÈTRES : TOUTES LES FORMES

```
def f(a, b=2, *args, c, d=4, **kwargs):  
    return a, b, args, c, d, kwargs  
  
f(1, c=3)           → (1, 2, (), 3, 4, {})  
f(1, 5, 6, 7, c=3, e=8) → (1, 5, (6, 7), 3, 4, {'e': 8})  
nombres, options = [1, 5], {'c': 3}  
f(*nombres, **options) → (1, 5, (), 3, 4, {})  
f(1)                 → TypeError ❶
```

a	positionnel ou nommé
b=2	valeur par défaut
*args	positionnels en surplus, reçus dans un tuple
c, d=4	après *args (ou un * seul) : nommés obligatoirement
**kwargs	nommés en surplus, reçus dans un dict
def g(x, /, y):	x positionnel uniquement [3.8+]
def h(x, *, y):	y nommé uniquement
f(*liste, **dico)	à l'appel : déballer une liste, un dict

PORTÉE DES VARIABLES

Un nom est cherché dans l'ordre **Local** → fonction **Englobante** → **Global** (module) → **Built-in** (fonctions natives) : règle **LEGB**.

```
compteur = 0  
def incrementer():  
    global compteur # sans cela : UnboundLocalError  
    compteur += 1  
incrementer()  
compteur           → 1  
  
def fabrique():  
    total = 0  
    def ajouter(n):  
        nonlocal total # variable de la fonction englobante  
        total += n  
        return total  
    return ajouter # une « fermeture » (closure)  
acc = fabrique()  
acc(5), acc(3)     → (5, 8)
```

Préférer paramètres et valeurs de retour à **global** : le code reste lisible et testable.

MODULES ET IMPORT

```
import math           # puis : math.sqrt(2)  
from math import sqrt, pi # puis : sqrt(2)  
import datetime as dt # alias : dt.date(2026, 1, 15)
```

- Un **module** est un fichier **.py** : **import outils** charge **outils.py** ; un **paquet** est un dossier de modules.
- Imports en tête de fichier : bibliothèque standard, puis paquets tiers, puis modules du projet.
- Éviter **from module import *** : on ne sait plus d'où viennent les noms.
- ❶ Ne pas nommer un fichier comme un module existant (**random.py**, **csv.py**...) : il masque le vrai.
- **if __name__ == "__main__"** : protège le code à n'exécuter qu'au lancement direct (page 1).
- Python cherche les modules dans le dossier du script, puis dans **sys.path**.

LAMBDA ET FONCTIONS EN ARGUMENT

```
double = lambda x: 2 * x # fonction anonyme  
double(4)           → 8  
mots = ["kiwi", "Ail", "banane"]  
sorted(mots, key=str.lower) → ['Ail', 'banane', 'kiwi']  
sorted(mots, key=lambda m: m[-1]) → ['banane', 'kiwi', 'Ail']  
ops = {"+": lambda a, b: a + b, "-": lambda a, b: a * b}  
ops["*"](3, 4)       → 12
```

Une fonction est un objet : on peut la ranger dans une variable, un dict, la passer en argument. PEP 8 : **lambda** pour une expression courte passée en argument, **def** sinon.

PIÈGE : VALEUR PAR DÉFAUT MODIFIABLE

```
def ajouter(x, L=[]): # Liste créée UNE fois  
    L.append(x)  
    return L  
ajouter(1)           → [1]  
ajouter(2)           → [1, 2] ❶  
  
def ajouter(x, L=None): # La bonne forme  
    if L is None:  
        L = [] # nouvelle liste à chaque appel  
    L.append(x)  
    return L  
ajouter(1), ajouter(2) → ([1], [2])
```

La valeur par défaut est évaluée **une seule fois**, à la définition de la fonction.

RÉCURSIVITÉ

```
def factorielle(n):  
    if n <= 1: # cas de base : indispensable  
        return 1  
    return n * factorielle(n - 1)  
factorielle(5) → 120  
import sys  
sys.getrecursionlimit() → 1000
```

Au-delà de la limite (1 000 appels imbriqués par défaut) : **RecursionError**. Une boucle est souvent plus sûre.

DÉCORATEURS

```
import functools, time  
  
def chronometrer(f):  
    @functools.wraps(f) # conserve nom et docstring  
    def enveloppe(*args, **kwargs):  
        t0 = time.perf_counter()  
        resultat = f(*args, **kwargs)  
        duree = time.perf_counter() - t0  
        print(f"{f.__name__} : {duree:.3f} s")  
        return resultat  
    return enveloppe  
  
@chronometrer # somme = chronometrer(somme)  
def somme(n):  
    return sum(range(n))  
  
@functools.cache # mémorise les résultats  
def fib(n):  
    return n if n < 2 else fib(n - 1) + fib(n - 2)  
fib(80)           → 23416728348467685
```

Un décorateur prend une fonction et en renvoie une autre qui l'enveloppe : **chronométré**, **journaliser**, **mettre en cache**, **contrôler l'accès**.
@functools.lru_cache(maxsize=128) : cache de taille bornée.

ANNOTATIONS DE TYPE

```
def moyenne(notes: list[float]) -> float:  
    return sum(notes) / len(notes)  
moyenne([12.0, 15.0]) → 13.5  
  
age: int = 36  
nom: str | None = None # « ou » entre types (3.10+)  
type Point = tuple[float, float] # alias de type (3.12+)  
moyenne("abc") → TypeError
```

Python **ne vérifie pas** les annotations à l'exécution (l'erreur ci-dessus vient de **sum**) : elles documentent et servent aux outils externes (**mypy**, **pyright**) et aux éditeurs.

6 · Classes et objets

DÉFINIR UNE CLASSE

```
class Compte:
    """Compte bancaire simple."""
    nb_comptes = 0 # attribut de classe

    def __init__(self, titulaire, solde=0):
        self.titulaire = titulaire # attributs d'instance
        self.solde = solde
        Compte.nb_comptes += 1

    def deposter(self, montant): # méthode
        if montant <= 0:
            raise ValueError("montant positif attendu")
        self.solde += montant
        return self.solde

    def __repr__(self): # affichage technique
        return f"Compte({self.titulaire!r}, {self.solde})"

c = Compte("Alice", 100) # crée l'objet, puis appelle __init__
c.deposer(50)            → 150
c.solde                  → 150
c                         → Compte('Alice', 150)
Compte.nb_comptes        → 1
```

classe	le modèle : Compte
instance, objet	un exemplaire : c
attribut	une donnée de l'objet : c.solde
méthode	une fonction de la classe : c.deposer(50) équivaut à Compte.deposer(c, 50)
self	l'instance concernée ; premier paramètre de chaque méthode (nom conventionnel)
__init__	initialise l'objet juste après sa création
__solde	usage interne (convention) · nom « brouillé » en __Compte__solde

HÉRITAGE

```
class CompteEpargne(Compte): # hérite de Compte
    def __init__(self, titulaire, solde=0, plafond=10_000):
        super().__init__(titulaire, solde) # Le parent d'abord
        self.plafond = plafond

    def deposter(self, montant): # redéfinition
        if self.solde + montant > self.plafond:
            raise ValueError("plafond dépassé")
        return super().deposer(montant)

e = CompteEpargne("Bob", 9_900)
e.deposer(50)            → 9950
e.deposer(100)           → ValueError: plafond dépassé
isinstance(e, Compte)    → True
[k.__name__ for k in CompteEpargne.__mro__]
→ ['CompteEpargne', 'Compte', 'object']
```

super() appelle la version de la classe parente, selon l'ordre de résolution des méthodes (**__mro__**). Héritage = relation « **est un** » ; pour « **a un** », préférer la **composition** : un attribut qui contient un autre objet.

TOUT EST OBJET : INTROSPECTION

type(42)	<class 'int'> · classe de l'objet
isinstance(True, int)	True · tient compte de l'héritage
hasattr("", "upper")	True · possède cet attribut ?
getattr(3, "real", None)	3 · attribut par son nom, avec défaut
callable(len)	True · callable ?
dir(x) · vars(x) · id(x)	attributs · dict des attributs · identité

Typage canard : peu importe la classe d'un objet, pourvu qu'il possède les méthodes attendues.

ÉNUMÉRATIONS

```
from enum import Enum, auto
class Couleur(Enum):
    ROUGE = auto()
    VERT = auto()
Couleur.VERT          → <Couleur.VERT: 2>
Couleur.VERT.name     → 'VERT'
Couleur(1)            → <Couleur.ROUGE: 1>
[c.name for c in Couleur] → ['ROUGE', 'VERT']
```

PROPRIÉTÉS, MÉTHODES DE CLASSE ET STATIQUES

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius # passe par le setter

    @property # Lecture : t.celsius
    def celsius(self):
        return self._celsius

    @celsius.setter # écriture
    def celsius(self, valeur):
        if valeur < -273.15:
            raise ValueError("sous le zéro absolu")
        self._celsius = valeur

    @property
    def fahrenheit(self): # attribut calculé
        return self._celsius * 9 / 5 + 32

    @classmethod # autre constructeur
    def depuis_fahrenheit(cls, f):
        return cls((f - 32) * 5 / 9)

    @staticmethod # ni self ni cls
    def gele(celsius):
        return celsius <= 0

Temperature(25).fahrenheit      → 77.0
Temperature.depuis_fahrenheit(212).celsius → 100.0
Temperature.gele(-3)            → True
Temperature(-300)               → ValueError: sous le zéro absolu
```

MÉTHODES SPÉCIALES

Elles branchent un objet sur la syntaxe du langage.

__init__(self, ...)	C(...) : initialisation
__repr__(self)	repr(x) , interpréteur : non ambigu
__str__(self)	str(x) , print(x) : lisible
__eq__(self, o)	x == y
__lt__(self, o)	x < y , donc sorted()
__hash__(self)	clé de dict, élément de set
__len__(self)	len(x)
__bool__(self)	if x:
__getitem__(self, i)	x[i]
__contains__(self, v)	v in x
__iter__(self)	for v in x:
__add__(self, o)	x + y ; aussi __sub__ , __mul__ ...
__call__(self, ...)	x(...) : objet callable
__enter__ , __exit__	with x: : gestionnaire de contexte

CLASSES DE DONNÉES : DATACLASSES

```
from dataclasses import dataclass, field, asdict

@dataclass(frozen=True) # frozen : immuable, hachable
class Point:
    x: float
    y: float = 0.0

p = Point(1.5)
p                                → Point(x=1.5, y=0.0)
p == Point(1.5, 0)              → True
asdict(p)                       → {'x': 1.5, 'y': 0.0}
p.x = 2                          → FrozenInstanceError !

@dataclass(order=True) # order : <, >, sorted()
class Tache:
    priorite: int
    titre: str = ""
    tags: list[str] = field(default_factory=list)
    sorted([Tache(2, "b"), Tache(1, "a")][0].titre → 'a'

@dataclass écrit pour vous __init__, __repr__ et __eq__. Valeur par défaut modifiable : field(default_factory=list).
```

7 · Erreurs, exceptions et fichiers

GÉRER LES EXCEPTIONS

```
def diviser(texte):
    try:
        n = int(texte)          # peut lever ValueError
        r = 100 / n            # peut lever ZeroDivisionError
    except ValueError:
        print("pas un entier")
    except ZeroDivisionError as e:  # e : L'objet exception
        print("erreur :", e)
    else:
        print("résultat :", r)    # seulement si aucune exception
    finally:
        print("fin")             # dans tous les cas

diviser("4")      → résultat : 25.0  fin
diviser("0")      → erreur : division by zero  fin
diviser("x")      → pas un entier  fin
```

LEVER UNE EXCEPTION

```
class SoldeInsuffisant(Exception):  # exception personnalisée
    """Le solde ne couvre pas le retrait."""

def retirer(solde, montant):
    if montant > solde:
        raise SoldeInsuffisant(f"il manque {montant - solde} €")
    return solde - montant

try:
    retirer(50, 70)
except SoldeInsuffisant as e:
    print("Refusé :", e)          → Refusé : il manque 20 €
```

- Attraper des exceptions **précises** ; jamais de **except** : nu, qui avale tout, même **Ctrl+C**.
- **raise** seul, dans un **except** : relance l'exception en cours ; **raise Autre("...") from e** : nouvelle exception, cause conservée.
- **except (ValueError, TypeError)** : plusieurs types ; **3.14+** parenthèses facultatives sans **as**.
- **assert x > 0, "message"** : contrôle de développement, ignoré avec **python -O**.

HIÉRARCHIE DES EXCEPTIONS

```
BaseException
+-- KeyboardInterrupt      Ctrl+C
+-- SystemExit              sys.exit()
+-- Exception               celles qu'on attrape
    +-- ArithmeticError
    |   +-- ZeroDivisionError
    +-- LookupError
    |   +-- IndexError       indice hors limites
    |   +-- KeyError         clé absente
    +-- NameError
    |   +-- UnboundLocalError
    +-- ImportError
    |   +-- ModuleNotFoundError
    +-- OSError              système, fichiers
    |   +-- FileNotFoundError
    |   +-- PermissionError
    +-- RuntimeError
    |   +-- RecursionError
    +-- AttributeError, ValueError...
```

except LookupError : attrape à la fois **IndexError** et **KeyError**.

LIRE UN MESSAGE D'ERREUR

```
def moyenne(notes):
    return sum(notes) / len(notes)
print(moyenne([]))
```

```
Traceback (most recent call last):
  File "moyenne.py", line 3, in <module>
    print(moyenne([]))
    ~~~~~^~~~~
  File "moyenne.py", line 2, in moyenne
    return sum(notes) / len(notes)
    ~~~~~^~~~~
ZeroDivisionError: division by zero
```

- 1 lire d'abord la **dernière ligne** : le type d'erreur et son message ;
 - 2 remonter : chaque « File ... line ... » est un appel ; le plus **bas** est l'endroit où l'erreur s'est produite ;
 - 3 les ~ et ^ soulignent l'expression fautive **3.11+**.
- Chemin du fichier abrégé ; la dernière ligne se copie telle quelle dans un moteur de recherche.

ERREURS FRÉQUENTES

SyntaxError : expected ':'	deux-points oublié après if , for , def ...
IndentationError : expected an indented block...	bloc non indenté ; tabulations et espaces mélangés
NameError : name 'nmbre' is not defined	faute de frappe, variable jamais définie, import oublié
TypeError : can only concatenate str (not "int") to str	convertir : str(36) , ou une f-string
TypeError : 'NoneType' object is not subscriptable	une fonction a renvoyé None
ValueError : invalid literal for int() with base 10: '12,5'	texte non convertible
IndexError : list index out of range	indice ≥ len(L)
KeyError : 'nom'	clé absente : d.get("nom")
AttributeError : 'list' object has no attribute 'push'	méthode inexistante : dir(x)
UnboundLocalError : cannot access local variable 'x'...	affectation dans la fonction : global ou paramètre
FileNotFoundError : [Errno 2] No such file or directory: 'absent.txt'	chemin relatif au dossier courant (Path.cwd())
ModuleNotFoundError : No module named 'paquet_absent'	non installé dans cet environnement : python -m pip install ...
UnicodeDecodeError : 'utf-8' codec can't decode byte 0xe9...	mauvais encodage : préciser encoding=

FICHIERS TEXTE

```
# écrire : Le fichier est créé, ou ÉCRASÉ s'il existe
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("12\n")
    print("15", file=f)          # print vers un fichier
# ajouter à la fin
with open("notes.txt", "a", encoding="utf-8") as f:
    f.write("18\n")
# lire ligne à ligne ; fermeture automatique en fin de bloc
with open("notes.txt", encoding="utf-8") as f:
    notes = [int(ligne) for ligne in f]
notes                                → [12, 15, 18]
```

"r"	lecture (défaut)	"w"	écriture, écrase ❶
"a"	ajout en fin	"x"	création seule
"rb"	"wb" binaire	f.read()	tout le texte
f.readline()	ligne suivante	f.readlines()	liste des lignes
f.write(s)	sans \n ajouté	print(..., file=f)	avec \n

Toujours préciser **encoding="utf-8"** : jusqu'à 3.14, le défaut sous Windows dépend du système (souvent **cp1252**) ; UTF-8 devient le défaut avec Python 3.15 (PEP 686).

PATHLIB : LES CHEMINS

```
from pathlib import Path
p = Path("donnees") / "notes.txt"  # / assemble les chemins
p.name, p.stem, p.suffix           → ('notes.txt', 'notes', '.txt')
p.parent.name                      → 'donnees'
p.parent.mkdir(parents=True, exist_ok=True)
p.write_text("12\n15\n", encoding="utf-8")      → 6
p.read_text(encoding="utf-8").split()           → ['12', '15']
p.exists()                                       → True
[f.name for f in Path("donnees").glob("*.txt")] → ['notes.txt']
```

Aussi : **Path.cwd()** dossier courant, **Path.home()** dossier personnel, **p.resolve()** chemin absolu, **rglob("*.py")** recherche récurive, **p.rename(q)**, **p.unlink()** suppression ❶. Sous Windows : **Path(r"C:\Users\...")** ou barres obliques **"C:/Users/..."**.

8 · Environnements, données, bibliothèque standard

MÉMENTO — PYTHON

ENVIRONNEMENTS VIRTUELS ET PIP

```
python -m venv          créer l'environnement
                        du projet (une fois)
.venv\Scripts\activate
                        activer : Windows (PowerShell :
                        Activate.ps1)
source                  activer : Linux, macOS
.venv/bin/activate
deactivate              désactiver
python -m pip           installer depuis PyPI
install paquet
pip install -U          mettre à jour
paquet
pip list · pip          paquets installés ·
show paquet            détails
pip freeze >           figer les versions
requirements.txt
pip install -r          réinstaller à l'identique
requirements.txt
```

- Un environnement **par projet** : les versions ne se contredisent pas d'un projet à l'autre.
- **python -m pip** installe pour ce Python-là, sans ambiguïté.
- **!** Vérifier le nom exact d'un paquet sur pypi.org : des paquets malveillants imitent les noms connus.
- Si PowerShell refuse l'activation : **Get-Help about_Execution_Policies**.
- Outils tiers répandus : uv, Poetry, pipx.

BIBLIOTHÈQUE STANDARD : L'ESSENTIEL

os · sys	système : os.environ , sys.argv , sys.exit()
pathlib · shutil · zipfile	chemins ; copier, déplacer des dossiers ; archives
datetime · time · zoneinfo	dates, durées, pauses (time.sleep), fuseaux
math · random · statistics	calcul, hasard, moyenne, médiane, écart type
decimal · fractions	calcul exact
collections · itertools · functools	structures et outils d'itération, voir ci-dessous
re	expressions régulières
json · csv · tomlib	formats de données (TOML : [3.11+])
sqlite3	base de données SQL dans un simple fichier
logging	journal d'exécution, au lieu de print
argparse	options de la ligne de commande
subprocess	lancer d'autres programmes
urllib.request	requêtes HTTP simples
unittest · doctest	tests automatiques
secrets · hashlib	jetons sûrs ; empreintes (hachage)
tkinter	interfaces graphiques simples

- **!** **random** ne convient pas à la sécurité : pour un mot de passe ou un jeton, **secrets**.

JSON ET CSV

```
import json
d = {"nom": "Élise", "notes": [12, 15]}
s = json.dumps(d, ensure_ascii=False)
s
→ '{"nom": "Élise", "notes": [12, 15]}'
json.loads(s) == d → True
with open("d.json", "w",
            encoding="utf-8") as f:
    json.dump(d, f, ensure_ascii=False,
              indent=2)
with open("d.json",
            encoding="utf-8") as f:
    d = json.load(f)

import csv
with open("n.csv", "w", newline="",
            encoding="utf-8") as f:
    w = csv.writer(f, delimiter=";")
    w.writerow(["nom", "note"])
    w.writerow(["Élise", 15])
with open("n.csv", newline="",
            encoding="utf-8") as f:
    lignes = list(csv.DictReader(
        f, delimiter=";"))
lignes[0]
→ {'nom': 'Élise', 'note': '15'}
```

JSON ↔ Python : objet ↔ **dict**, tableau ↔ **list**, **true** / **null** ↔ **True** / **None**. CSV : tout est lu comme **texte** ('15'); **newline=""** obligatoire. Excel en français attend souvent ; et lit mieux les accents en **encoding="utf-8-sig"**.

COLLECTIONS ET ITERTOOLS

```
from collections import Counter
Counter([1, 2, 2, 3, 2]).most_common(1)
→ [(2, 3)]
from collections import defaultdict
groupes = defaultdict(list)
groupes["a"].append(1) # clé créée
dict(groupes)
→ {'a': [1]}
from collections import deque
q = deque([1, 2], maxlen=3)
q.appendleft(0)
q.append(3) # 3 au plus
q
→ deque([1, 2, 3], maxlen=3)
q.popleft() → 1
from itertools import combinations
list(combinations("abc", 2))
→ [('a', 'b'), ('a', 'c'), ('b', 'c')]
from itertools import permutations
len(list(permutations("abcd"))) → 24
from itertools import product
list(product([0, 1], repeat=2))
→ [(0, 0), (0, 1), (1, 0), (1, 1)]
from itertools import accumulate
list(accumulate([1, 2, 3, 4]))
→ [1, 3, 6, 10]
```

DATES ET HEURES

```
from datetime import date, datetime
from datetime import timedelta
d = date(2026, 1, 15)
d + timedelta(days=30)
→ datetime.date(2026, 2, 14)
(datetime(2026, 12, 25) - d).days → 344
d.weekday() → 3
d.strftime("%d/%m/%Y") → '15/01/2026'
d.isoformat() → '2026-01-15'
datetime.strptime("15/01/2026 14:30",
                  "%d/%m/%Y %H:%M")
→ datetime.datetime(2026, 1, 15, 14, 30)

weekday() : lundi = 0. date.today(),
datetime.now() : aujourd'hui, maintenant. Codes :
%Y année, %m mois, %d jour, %H %M %S heure, minute,
seconde, %A nom du jour (anglais par défaut).
Fuseaux : ZoneInfo("Europe/Paris") du module
zoneinfo ; sous Windows, installer d'abord le
paquet tzdata.
```

EXPRESSIONS RÉGULIÈRES : RE

.	tout caractère	\d	chiffre
\w	lettre, chiffre, _	\s	blanc
[aeiou]	l'un de	[^0-9]	sauf
*	0 ou plus	+	1 ou plus
?	0 ou 1	{2,4}	de 2 à 4
^ \$	début, fin	\b	limite de mot
()	groupe		ou
*?	non gourmand	\.	point littéral

```
import re
re.findall(r"\d+", "a1b22c333")
→ ['1', '22', '333']
re.sub(r"\s+", " ", "a b\t c")
→ 'a b c'
motif = r"(\d\d)/(\d\d)/(\d{4})"
m = re.search(motif, "le 15/01/2026")
m.group(3) → '2026'
bool(re.fullmatch(r"[A-Z]\d+", "B123"))
→ True
```

Toujours des chaînes brutes **r"..."**. **search** cherche partout, **match** au début, **fullmatch** sur toute la chaîne.

TESTER, DÉBOGUEUR, MESURER

breakpoint()	pause dans le débogueur : [n] ligne suivante, [S] entrer, [C] continuer, p x afficher, [q] quitter
python -m pdb prog.py	déboguer dès la première ligne
print(f"{x=}")	affichage de débogage
python -m unittest	lancer les tests unittest
python -m pytest	lancer les tests pytest (paquet tiers)
python -m timeit "sum(range(1000))"	mesurer une durée
logging.basicConfig(level=logging.INFO)	journal avec niveaux

```
# fichier test_calcul.py
# Lancer : python -m pytest
from calcul import moyenne

def test_moyenne():
    assert moyenne([10, 20]) == 15
```

STYLE : PEP 8 ET BONNES PRATIQUES

- 4 espaces par niveau ; lignes de 79 caractères au plus.
- **snake_case** pour variables et fonctions, **CapWords** pour les classes, **MAJUSCULES** pour les constantes.
- Deux lignes vides entre fonctions et classes de premier niveau.
- **is None** plutôt que **== None** ; **if liste** : plutôt que **if len(liste) > 0** ;
- **with** pour les fichiers ; **f-strings** pour mettre en forme.
- Fonctions courtes, noms explicites, docstrings.
- Formatage et analyse automatiques : Ruff, Black ; types : mypy (outils tiers).
- **import this** : les principes de Python (PEP 20).